AutoDamage-Vision: End-to-End Vehicle Damage Detection and Severity Classification Using Deep Learning

Abstract

Vehicle damage assessment is a critical task in automobile insurance, repair estimation, and fleet management. Manual inspection is time-consuming, subjective, and costly. This project presents **AutoDamage-Vision**, an end-to-end computer vision pipeline that automatically detects vehicle damage regions and classifies their severity using deep learning. The system employs a **YOLOv8-based object detector** to localize damage areas and a **Convolutional Neural Network (CNN)** to classify the severity of detected damage into three levels: *Minor, Moderate, and Severe*. The modular design allows independent optimization of detection and classification stages. Experimental results demonstrate that the proposed approach achieves reasonable detection performance with an mAP@50 of approximately **0.43** and a severity classification accuracy exceeding **90% on training data**, highlighting both the feasibility and challenges of automated damage assessment.

1. Introduction

Vehicle damage assessment plays a vital role in insurance claim processing, accident analysis, and vehicle maintenance. Traditional damage inspection relies heavily on human experts, making the process slow, subjective, and inconsistent across evaluators. With the rapid growth of computer vision and deep learning, automated damage detection systems have become increasingly viable.

The objective of this project is to design and implement an **automated vehicle damage analysis pipeline** capable of:

1. Detecting damage regions in vehicle images.
2. Classifying the severity of each detected damage region.

Instead of training a single monolithic model, this project adopts a **modular computer vision pipeline**, where object detection and severity classification are handled by separate models. This approach mirrors real-world deployment systems and allows independent improvements to each component.

2. Dataset Description

The dataset used in this project is the **Vehicle Damage Detection Dataset** sourced from Kaggle. It consists of real-world vehicle images containing various types of damages captured under different lighting conditions, viewpoints, and backgrounds.

2.1 Dataset Characteristics

- Total images: Over **11,000**
- Damage categories include:
    - Dents
    - Scratches
    - Broken glass
    - Paint damage
    - Deformations
- Images vary in resolution and quality
- Multiple damage regions may exist within a single image

2.2 Challenges

Several challenges were observed during dataset preparation:

- **Class imbalance**, where certain damage types appear more frequently.
- **Annotation noise**, including bounding boxes extending beyond image boundaries.
- **Corrupt images and labels**, which had to be ignored during validation.
- **Severity ambiguity**, as severity is a subjective concept and not solely dependent on bounding box size.

3. Object Detection Using YOLOv8

3.1 Model Selection

YOLOv8 was selected as the object detection model due to:

- Its real-time performance
- Clean training and evaluation APIs
- Industry adoption
- Strong performance on object localization tasks

3.2 Dataset Preparation

Annotations were converted into YOLO format:

```
<class_id> <x_center> <y_center> <width> <height>
```

All values were normalized to the image dimensions. Images were split into training and validation sets.

3.3 Training Configuration

- Image size: 640 × 640
- Epochs: 30
- Optimizer: Default YOLOv8 optimizer
- Hardware: NVIDIA Tesla T4 GPU

3.4 Evaluation Metrics

The model was evaluated using standard object detection metrics:

- **Precision**
- **Recall**
- **mAP@50**

- **mAP@50–95**

4. Detection Results and Error Analysis

4.1 Overall Performance

The trained YOLOv8 model achieved the following validation performance:

- Precision: **0.52**
- Recall: **0.42**
- mAP@50: **0.43**
- mAP@50–95: **0.26**

These results indicate that the model is able to localize most prominent damage regions, though recall remains moderate due to missed detections.

4.2 Class-wise Observations

- Damage involving broken glass performed best due to clear visual cues.
- Small scratches and paint damage were harder to detect.
- Overlapping damages caused partial localization failures.

4.3 Error Sources

- Non-normalized bounding boxes in some labels
- Truncated image files
- Small object sizes
- Occlusion and lighting variations

Despite these issues, the model provided a reliable foundation for downstream severity classification.

5. Damage Severity Classification Using CNN

5.1 Motivation

Instead of embedding severity prediction into the detector, a separate CNN classifier was trained on cropped damage regions. This improves modularity and interpretability.

Severity levels:

- **Minor**: Small scratches, surface-level dents
- **Moderate**: Visible deformation
- **Severe**: Broken or detached parts

5.2 Dataset Construction

Bounding boxes predicted by YOLOv8 were used to crop damage regions. Cropped images were resized to **128 × 128** pixels and organized into class folders.

5.3 CNN Architecture

The CNN architecture consists of:

- Convolutional layers with ReLU activation
- Max-pooling layers
- Fully connected dense layers
- Softmax output layer for 3-class classification

5.4 Training Results

- Training accuracy: **~92%**
- Training loss steadily decreased
- Validation accuracy fluctuated between **30–40%**

This gap indicates **overfitting**, likely due to:

- Limited labeled severity data
- Subjective class boundaries
- Visual similarity between severity levels

6. End-to-End Pipeline Integration

The final system integrates both models into a single inference pipeline:

1. Input image is passed to YOLOv8
2. Damage regions are detected
3. Each bounding box is cropped
4. Cropped region is resized and normalized
5. CNN predicts severity
6. Bounding box and severity label are visualized

This modular design ensures:

- Errors in detection do not break the classifier
- Each model can be retrained independently
- The system resembles real-world deployment pipelines

7. Results and Discussion

7.1 Strengths

- Fully automated pipeline
- Real-world dataset

- Clean separation of detection and classification
- Visual interpretability

7.2 Limitations

- Severity classification suffers from label ambiguity
- Detection errors propagate to classification
- Limited validation performance for severity classifier

7.3 Key Insight

High training accuracy alone is not sufficient. Understanding *why* models fail is more important than maximizing metrics.

8. Conclusion and Future Work

This project successfully demonstrates an end-to-end deep learning pipeline for vehicle damage detection and severity classification. While detection performance is satisfactory, severity classification remains challenging due to subjective labeling and limited data.